

Tutorials › Security › Cluster-Level-Pss

Apply Pod Security Standards at the Cluster Level | Kubernetes

Apply Pod Security Standards at the Cluster LevelNoteThis tutorial applies only for new clusters.Pod Security is an admission controller that carries out checks against the Kubernetes

Pod Security Standards when new pods are created. It is a feature GA'ed in v1.25.

This tutorial shows you how to enforce the baseline Pod Security Standard at the cluster level which applies a standard configuration

to all namespaces in a cluster.To apply Pod Security Standards to specific namespaces, refer to Apply Pod Security Standards at the namespace level.If you are running a version of Kubernetes other than v1.36,

check the documentation for that version.Before you beginInstall the following on your

workstation:kindkubectlThis tutorial demonstrates what you can configure for a Kubernetes cluster that you fully control. If you are learning how to configure Pod Security Admission for a managed cluster where you are not able to configure the control plane, read

Apply Pod Security Standards at the namespace level.Choose the right Pod Security Standard to applyPod Security Admission

lets you apply built-in Pod Security Standards

with the following modes: enforce, audit, and warn.To gather information that helps you to choose the Pod Security Standards

that are most appropriate for your configuration, do the following:Create a cluster with no Pod Security

Standards applied:kind create cluster --name psa-wo-cluster-pss

The output is similar to:Creating cluster "psa-wo-cluster-pss" ...

✓ Ensuring node image (kindest/node:v1.36.0) ■

✓ Preparing nodes ■

✓ Writing configuration ■

✓ Starting control-plane ■■

✓ Installing CNI ■

✓ Installing StorageClass ■

Set kubectl context to "kind-psa-wo-cluster-pss"

You can now use your cluster with:

```
kubectl cluster-info --context kind-psa-wo-cluster-pss
```

Thanks for using kind! ■

Set the kubectl context to the new cluster:kubectl cluster-info --context kind-psa-wo-cluster-pss

The output is similar to this:Kubernetes control plane is running at https://127.0.0.1:61350

CoreDNS is running at https://127.0.0.1:61350/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.

Get a list of namespaces in the cluster:kubectl get ns

The output is similar to this:NAME STATUS AGE

default Active 9m30s

kube-node-lease Active 9m32s

kube-public Active 9m32s

kube-system Active 9m32s

local-path-storage Active 9m26s

Use --dry-run=server to understand what happens when different Pod Security Standards

are applied:Privilegedkubectl label --dry-run=server --overwrite ns --all \

pod-security.kubernetes.io/enforce=privileged

The output is similar to:

```
namespace/default labeled
namespace/kube-node-lease labeled
namespace/kube-public labeled
namespace/kube-system labeled
namespace/local-path-storage labeled
Baselinekubectl label --dry-run=server --overwrite ns --all \
pod-security.kubernetes.io/enforce=baseline
```

The output is similar to:

```
namespace/default labeled
namespace/kube-node-lease labeled
namespace/kube-public labeled
Warning: existing pods in namespace "kube-system" violate the new PodSecurity enforce level "baseline:latest"
Warning: etcd-psa-wo-cluster-pss-control-plane (and 3 other pods): host namespaces, hostPath volumes
Warning: kindnet-vzj42: non-default capabilities, host namespaces, hostPath volumes
Warning: kube-proxy-m6hwf: host namespaces, hostPath volumes, privileged
namespace/kube-system labeled
namespace/local-path-storage labeled
Restrictedkubectl label --dry-run=server --overwrite ns --all \
pod-security.kubernetes.io/enforce=restricted
```

The output is similar to:

```
namespace/default labeled
namespace/kube-node-lease labeled
namespace/kube-public labeled
Warning: existing pods in namespace "kube-system" violate the new PodSecurity enforce level
"restricted:latest"
Warning: coredns-7bb9c7b568-hsptc (and 1 other pod): unrestricted capabilities, runAsNonRoot != true,
seccompProfile
Warning: etcd-psa-wo-cluster-pss-control-plane (and 3 other pods): host namespaces, hostPath volumes,
allowPrivilegeEscalation != false, unrestricted capabilities, restricted volume types, runAsNonRoot != true
Warning: kindnet-vzj42: non-default capabilities, host namespaces, hostPath volumes, allowPrivilegeEscalation
!= false, unrestricted capabilities, restricted volume types, runAsNonRoot != true, seccompProfile
Warning: kube-proxy-m6hwf: host namespaces, hostPath volumes, privileged, allowPrivilegeEscalation != false,
unrestricted capabilities, restricted volume types, runAsNonRoot != true, seccompProfile
namespace/kube-system labeled
Warning: existing pods in namespace "local-path-storage" violate the new PodSecurity enforce level
"restricted:latest"
Warning: local-path-provisioner-d6d9f7ffc-lw9lh: allowPrivilegeEscalation != false, unrestricted capabilities,
runAsNonRoot != true, seccompProfile
namespace/local-path-storage labeled
```

From the previous output, you'll notice that applying the privileged Pod Security Standard shows no warnings for any namespaces. However, baseline and restricted standards both have warnings, specifically in the kube-system namespace. Set modes, versions and standards

In this section, you apply the following Pod Security Standards to the latest version:

- baseline standard in enforce mode.
- restricted standard in warn and audit mode.

The baseline Pod Security Standard provides a convenient middle ground that allows keeping the exemption list short and prevents known privilege escalations. Additionally, to prevent pods from failing in kube-system, you'll exempt the namespace from having Pod Security Standards applied. When you implement Pod Security Admission in your own environment, consider the following:

- Based on the risk posture applied to a cluster, a stricter Pod Security Standard like restricted might be a better choice.
- Exempting the kube-system namespace allows pods to run as privileged in this namespace. For real world use, the Kubernetes project strongly recommends that you apply strict RBAC policies that limit access to kube-system, following the principle of least privilege.

To implement the preceding standards, do the following:

- Create a configuration file that can be consumed by the Pod Security Admission Controller to implement these Pod Security Standards:


```
mkdir -p /tmp/pss
```

```
cat <<EOF > /tmp/pss/cluster-level-pss.yaml
apiVersion: apiserver.config.k8s.io/v1
kind: AdmissionConfiguration
plugins:
- name: PodSecurity
configuration:
apiVersion: pod-security.admission.config.k8s.io/v1
kind: PodSecurityConfiguration
defaults:
enforce: "baseline"
enforce-version: "latest"
audit: "restricted"
audit-version: "latest"
warn: "restricted"
warn-version: "latest"
exemptions:
usernames: []
runtimeClasses: []
namespaces: [kube-system]
EOF
```

Note:pod-security.admission.config.k8s.io/v1 configuration requires v1.25+.

For v1.23 and v1.24, use v1beta1.

For v1.22, use v1alpha1.Configure the API server to consume this file during cluster creation:cat <<EOF >
/tmp/pss/cluster-config.yaml

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
kubeadmConfigPatches:
- |
kind: ClusterConfiguration
apiServer:
extraArgs:
admission-control-config-file: /etc/config/cluster-level-pss.yaml
extraVolumes:
- name: accf
hostPath: /etc/config
mountPath: /etc/config
readOnly: false
pathType: "DirectoryOrCreate"
extraMounts:
- hostPath: /tmp/pss
containerPath: /etc/config
# optional: if set, the mount is read-only.
# default false
readOnly: false
# optional: if set, the mount needs SELinux relabeling.
# default false
selinuxRelabel: false
# optional: set propagation mode (None, HostToContainer or Bidirectional)
# see https://kubernetes.io/docs/concepts/storage/volumes/#mount-propagation
# default None
propagation: None
EOF
```

Note:If you use Docker Desktop with kind on macOS, you can

add /tmp as a Shared Directory under the menu item

Preferences > Resources > File Sharing. Create a cluster that uses Pod Security Admission to apply these Pod Security Standards: `kind create cluster --name psa-with-cluster-pss --config`

`/tmp/pss/cluster-config.yaml`

The output is similar to this: `Creating cluster "psa-with-cluster-pss" ...`

✓ Ensuring node image (kindest/node:v1.36.0) ■

✓ Preparing nodes ■

✓ Writing configuration ■

✓ Starting control-plane ■■

✓ Installing CNI ■

✓ Installing StorageClass ■

Set kubectl context to "kind-psa-with-cluster-pss"

You can now use your cluster with:

`kubectl cluster-info --context kind-psa-with-cluster-pss`

Have a question, bug, or feature request? Let us know! <https://kind.sigs.k8s.io/#community> ■

Point kubectl to the cluster: `kubectl cluster-info --context kind-psa-with-cluster-pss`

The output is similar to this: `Kubernetes control plane is running at https://127.0.0.1:63855`

`CoreDNS is running at https://127.0.0.1:63855/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy`

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.

Create a Pod in the default namespace: `security/example-baseline-pod.yaml`

`apiVersion: v1`

`kind: Pod`

`metadata:`

`name: nginx`

`spec:`

`containers:`

`- image: nginx`

`name: nginx`

`ports:`

`- containerPort: 80`

`kubectl apply -f https://k8s.io/examples/security/example-baseline-pod.yaml`

The pod is started normally, but the output includes a warning: `Warning: would violate PodSecurity`

`"restricted:latest": allowPrivilegeEscalation != false (container "nginx" must set securityContext.allowPrivilegeEscalation=false), unrestricted capabilities (container "nginx" must set securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true (pod or container "nginx" must set securityContext.runAsNonRoot=true), seccompProfile (pod or container "nginx" must set securityContext.seccompProfile.type to "RuntimeDefault" or "Localhost")`

`pod/nginx created`

Clean up Now delete the clusters which you created above by running the following command: `kind delete`

`cluster --name psa-with-cluster-pss`

`kind delete cluster --name psa-wo-cluster-pss`

What's next Run a

shell script

to perform all the preceding steps at once: Create a Pod Security Standards based cluster level

Configuration Create a file to let API server consume this configuration Create a cluster that creates an API

server with this configuration Set kubectl context to this new cluster Create a minimal pod yaml file Apply this file

to create a Pod in the new cluster Pod Security Admission Pod Security Standards Apply Pod Security Standards

at the namespace level Feedback Was this page helpful? Yes

No Thanks for the feedback. If you have a specific, answerable question about how to use Kubernetes, ask it on Stack Overflow.

Open an issue in the GitHub Repository if you want to

report a problem

or

suggest an improvement. Last modified November 30, 2023 at 1:22 AM PST: [en] Update cluster-level-pss.md
show the code sample what user will apply (310a1221ac)